

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop **Code:**

```
#include <stdio.h>

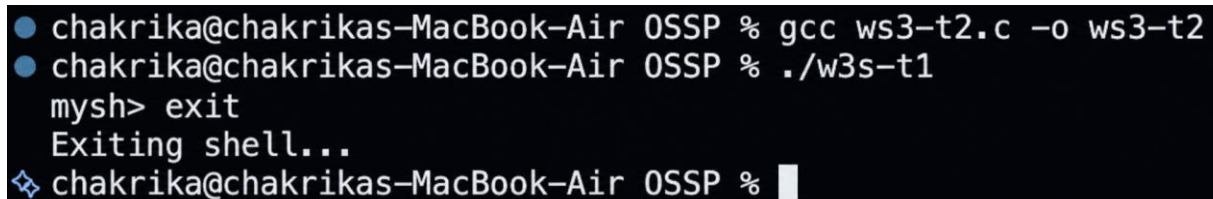
#include <string.h> #define MAX 256 int main() {
char input[MAX]; while (1) { printf("mysh>
"); fflush(stdout); if (fgets(input,
sizeof(input), stdin) == NULL) {
printf("\n"); break;
}
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
continue;

if (strcmp(input, "exit") == 0) {
printf("Exiting shell...\n"); break;
}

printf("You entered: %s\n", input);
}
return 0;
}
```

Output:

A terminal window with a black background and white text. The prompt is 'chakrika@chakrikas-MacBook-Air OSSP %'. The user enters 'gcc ws3-t2.c -o ws3-t2'. The prompt is 'chakrika@chakrikas-MacBook-Air OSSP %'. The user enters './ws3-t1'. The prompt is 'mysh>'. The user enters 'exit'. The output is 'Exiting shell...'. The prompt is 'chakrika@chakrikas-MacBook-Air OSSP %' followed by a cursor.

```
● chakrika@chakrikas-MacBook-Air OSSP % gcc ws3-t2.c -o ws3-t2
● chakrika@chakrikas-MacBook-Air OSSP % ./ws3-t1
mysh> exit
Exiting shell...
❖ chakrika@chakrikas-MacBook-Air OSSP %
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close() open()**

syntax: open(" filename.txt", modes, permissions)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

```
#include <unistd.h>
int main() {   char
buffer[100];   int
n;
```

```

    printf("Enter some text: ");    n = read(0,
buffer, sizeof(buffer) - 1);    buffer[n] = '\0';
// Null-terminate the string    printf("You
entered: %s", buffer);
    return 0; }

```

write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

```
#include <unistd.h>
```

```
int main()
```

```

{
    write(1, "Hello World\n", 12);
    return 0;
} Code:

```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#define BUF_SIZE 128
```

```
int main() {    int fd1, fd2,
```

```
n;                char
```

```
buffer[BUF_SIZE];
```

```

    fd1 = open("File1.txt", O_RDONLY);    if (fd1
< 0) { perror("open File1.txt"); exit(1); }

```

```

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
if (fd2 < 0) { perror("open File2.txt"); close(fd1); exit(1); }

    while ((n = read(fd1, buffer, BUF_SIZE)) > 0) {
        if (write(fd2, buffer, n) != n) {
perror("write");        close(fd1);
close(fd2);        exit(1);
        }
    }
    if (n < 0) perror("read");

    close(fd1);
close(fd2);

    printf("File1.txt copied to File2.txt successfully.\n");
return 0;
}

```

Output:

```

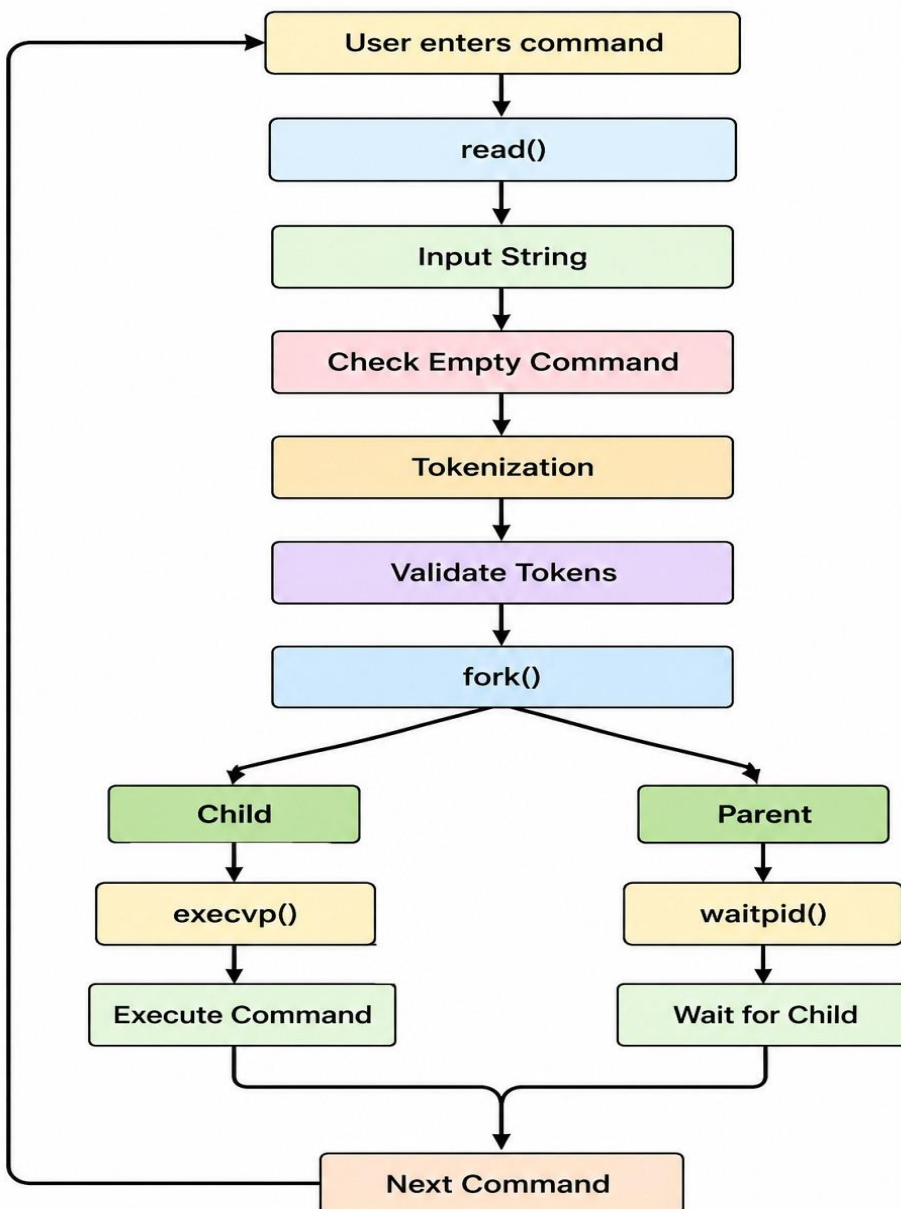
❌ chakrika@Chakrikas-MacBook-Air 0SSP % ./w3s-t2
open File1.txt: No such file or directory
❖ chakrika@Chakrikas-MacBook-Air 0SSP % █

```

Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures Flow Chart:



CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

void tokenize(char input[]) {
    char *token;
    int count = 0;

    printf("\n--- TOKENS ---\n");

    token = strtok(input, " \t\n");

    while (token != NULL) {
        printf("Token %d: %s\n", ++count, token);
        token = strtok(NULL, " \t\n");
    }

    if (count == 0)
        printf("Empty command\n");
}

int main() {
    char input[MAX];

    printf("=====\n");
    printf("  LEXICAL ANALYSIS AND PARSING\n");
    printf("=====\n");

    printf("Enter command: ");
    fgets(input, MAX, stdin);

    if (strlen(input) == 1) {
        printf("\nError: Empty command.\n");
        return 0;
    }

    tokenize(input);

    printf("\n--- PARSER ---\n");
```

```
printf("Syntax validation successful.\n");
printf("Parse completed successfully.\n");

printf("\n--- EXECUTION STRUCTURE ---\n");
printf("Input Command\n");
printf("    |\n");
printf("Tokenizer\n");
printf("    |\n");
printf("Parser\n");
printf("    |\n");
printf("Syntax Check\n");
printf("    |\n");
printf("Execution Structure\n");

return 0;
}
```

Output:

```

chakrika@Chakrika:~/OSSP$ ./task1
=====
      LEXICAL ANALYSIS AND PARSING
=====
Enter command: ls -l /home

--- TOKENS ---
Token 1: ls
Token 2: -l
Token 3: /home

--- PARSER ---
Syntax validation successful.
Parse completed successfully.

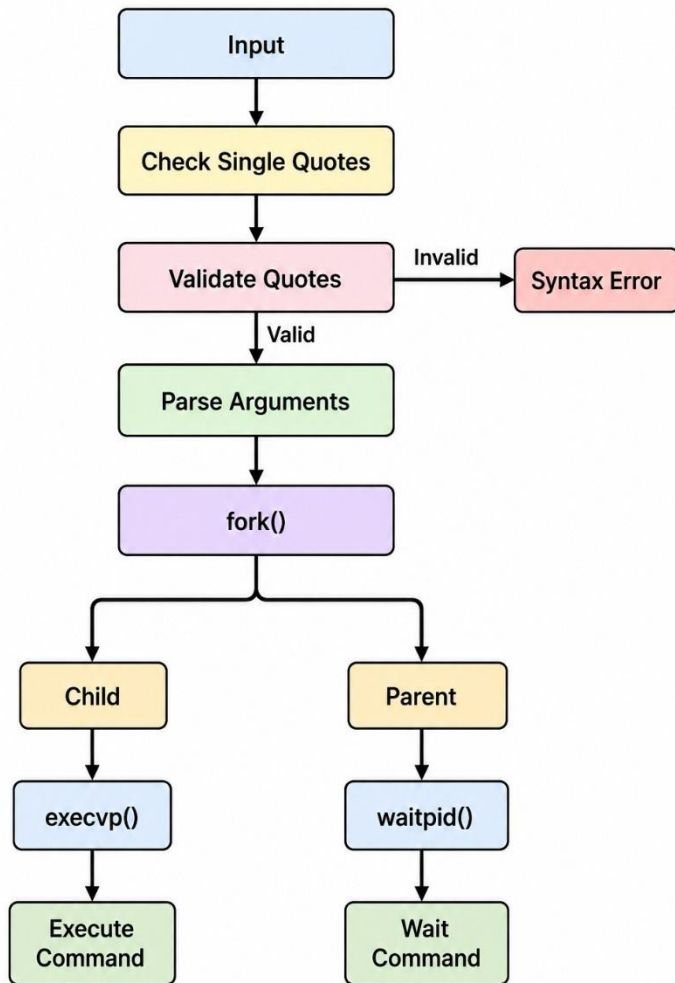
--- EXECUTION STRUCTURE ---
Input Command
    |
  Tokenizer
    |
  Parser
    |
Syntax Check
    |
Execution Structure
chakrika@Chakrika:~/OSSP$ |

```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:



Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Child PID: 3210 Hello

Command execution completed.

Case2: myShell> echo
'Hello World

Syntax Error: Unmatched single quote.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 50

int parse_command(char *input, char *args[]) {
    int argc = 0, i = 0, in_quotes = 0;
    char token[MAX_INPUT];
    int j = 0;

    while (input[i] != '\0') {
        if (input[i] == '"') {
            in_quotes = !in_quotes;
            i++;
            continue;
        }

        if ((input[i] == ' ' || input[i] == '\t' || input[i] == '\n') && !in_quotes) {
            if (j > 0) {
                token[j] = '\0';
                args[argc] = malloc(strlen(token) + 1);
                strcpy(args[argc], token);
                argc++;
                j = 0;
            }
            i++;
            continue;
        }

        token[j++] = input[i++];
    }

    if (in_quotes) {
        printf("\nSyntax Error: Unmatched single quote.\n");
        return -1;
    }
}
```

```

    }

    if (j > 0) {
        token[j] = '\0';
        args[argc] = malloc(strlen(token) + 1);
        strcpy(args[argc], token);
        argc++;
    }

    args[argc] = NULL;
    return argc;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("    MY SHELL - TASK 2\n");
    printf("    SINGLE QUOTE PARSING\n");
    printf("=====\n");
    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 1;
    }

    int argc = parse_command(input, args);

    if (argc == -1) {
        return 0;
    }

    if (argc == 0) {
        printf("\nSyntax Error: Empty command.\n");
        return 0;
    }

    printf("\nParsing Result:\n");

    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

```

```

    }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("Command execution completed.\n");
    }

    for (int i = 0; i < argc; i++) {
        free(args[i]);
    }

    return 0;
}

```

Output:

```

=====
MY SHELL - TASK 2
SINGLE QUOTE PARSING
=====
myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello
Child PID: 1652
Hello
Command execution completed.
chakrika@Chakrika:~/OSSP$ ./task2
=====
MY SHELL - TASK 2
SINGLE QUOTE PARSING
=====
myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World
Child PID: 1654
Hello World
Command execution completed.

```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Example Output:

Case1: myShell> echo "Hello
\$USER"

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process PID
= 3022

Hello raghu

Child process completed.

Case2: myShell> echo

"Hello World

Syntax Error: Unmatched double quote.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
#define MAX_INPUT 256
#define MAX_ARGS 50
```

```

int parse_command(char *input, char *args[]) {
    int argc = 0, i = 0, in_quotes = 0;
    char quote_type = 0;
    char token[MAX_INPUT];
    int j = 0;

    while (input[i] != '\0') {
        if (input[i] == '"') {
            if (!in_quotes) {
                in_quotes = 1;
                quote_type = '"';
            } else if (quote_type == '"') {
                in_quotes = 0;
                quote_type = 0;
            }
            i++;
            continue;
        }

        if ((input[i] == ' ' || input[i] == '\t' || input[i] == '\n') && !in_quotes) {
            if (j > 0) {
                token[j] = '\0';
                args[argc] = malloc(strlen(token) + 1);
                strcpy(args[argc], token);
                argc++;
                j = 0;
            }
            i++;
            continue;
        }

        token[j++] = input[i++];
    }

    if (in_quotes) {
        printf("\nSyntax Error: Unmatched double quote.\n");
        return -1;
    }

    if (j > 0) {
        token[j] = '\0';
        args[argc] = malloc(strlen(token) + 1);
        strcpy(args[argc], token);
        argc++;
    }

    args[argc] = NULL;
    return argc;
}

```

```

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("    MY SHELL - TASK 3\n");
    printf("  DOUBLE QUOTE PARSING\n");
    printf("=====\n");
    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    int argc = parse_command(input, args);

    if (argc == -1)
        return 0;

    if (argc == 0) {
        printf("\nSyntax Error: Empty command.\n");
        return 0;
    }

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < argc; i++)
        printf("Argument %d: %s\n", i + 1, args[i]);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process PID = %d\n", getpid());

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);
        printf("\nChild process completed.\n");
    }
}

```

```

    for (int i = 0; i < argc; i++)
        free(args[i]);

    return 0;
}

```

OUTPUT:

```

chakrika@Chakrika:~/OSSP$ ./task3
=====
                MY SHELL - TASK 3
            DOUBLE QUOTE PARSING
=====
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello $USER

Child Process PID = 1716
Hello $USER

Child process completed.

```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 256

```

```
#define MAX_ARGS 50
```

```
int parse_command(char *input, char *args[]) {  
    int argc = 0, i = 0;  
    int escaped = 0;  
    char token[MAX_INPUT];  
    int j = 0;  
  
    while (input[i] != '\0') {  
        if (escaped) {  
            token[j++] = input[i];  
            escaped = 0;  
            i++;  
            continue;  
        }  
  
        if (input[i] == '\\') {  
            escaped = 1;  
            i++;  
            continue;  
        }  
  
        if (input[i] == ' ' || input[i] == '\t' || input[i] == '\n') {  
            if (j > 0) {  
                token[j] = '\0';  
                args[argc] = malloc(strlen(token) + 1);  
                strcpy(args[argc], token);  
                argc++;  
                j = 0;  
            }  
            i++;  
            continue;  
        }  
  
        token[j++] = input[i++];  
    }  
  
    if (escaped) {
```

```

        token[j++] = '\\';
    }

    if (j > 0) {
        token[j] = '\0';
        args[argc] = malloc(strlen(token) + 1);
        strcpy(args[argc], token);
        argc++;
    }

    args[argc] = NULL;
    return argc;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("    MY SHELL - TASK 4\n");
    printf("  ESCAPE SEQUENCE PARSING\n");
    printf("=====\n");
    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    int argc = parse_command(input, args);

    if (argc == 0) {
        printf("\nSyntax Error: Empty command.\n");
        return 0;
    }

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < argc; i++)
        printf("Argument %d: %s\n", i + 1, args[i]);

```

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    printf("\nChild Process PID = %d\n", getpid());

    execvp(args[0], args);

    perror("execvp");
    exit(1);
} else {
    int status;
    waitpid(pid, &status, 0);

    printf("\nChild process completed.\n");
}

for (int i = 0; i < argc; i++)
    free(args[i]);

return 0;
}
```

OUTPUT:

CASE 1:

```
=====
      MY SHELL - TASK 4
      ESCAPE SEQUENCE PARSING
=====
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process PID = 1781
Hello World

Child process completed.

Child process completed.
```

CASE 2:

```
chakrika@Chakrika:~/OSSP$ ./task4
=====
      MY SHELL - TASK 4
      ESCAPE SEQUENCE PARSING
=====
myShell> echo Hello\&World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello&World

Child Process PID = 1813
Hello&World

Child process completed.
```

CASE 3:

```
chakrika@Chakrika:~/OSSP$ ./task4
=====
      MY SHELL - TASK 4
      ESCAPE SEQUENCE PARSING
=====
myShell> echo Hello\ World\ \&\ Test

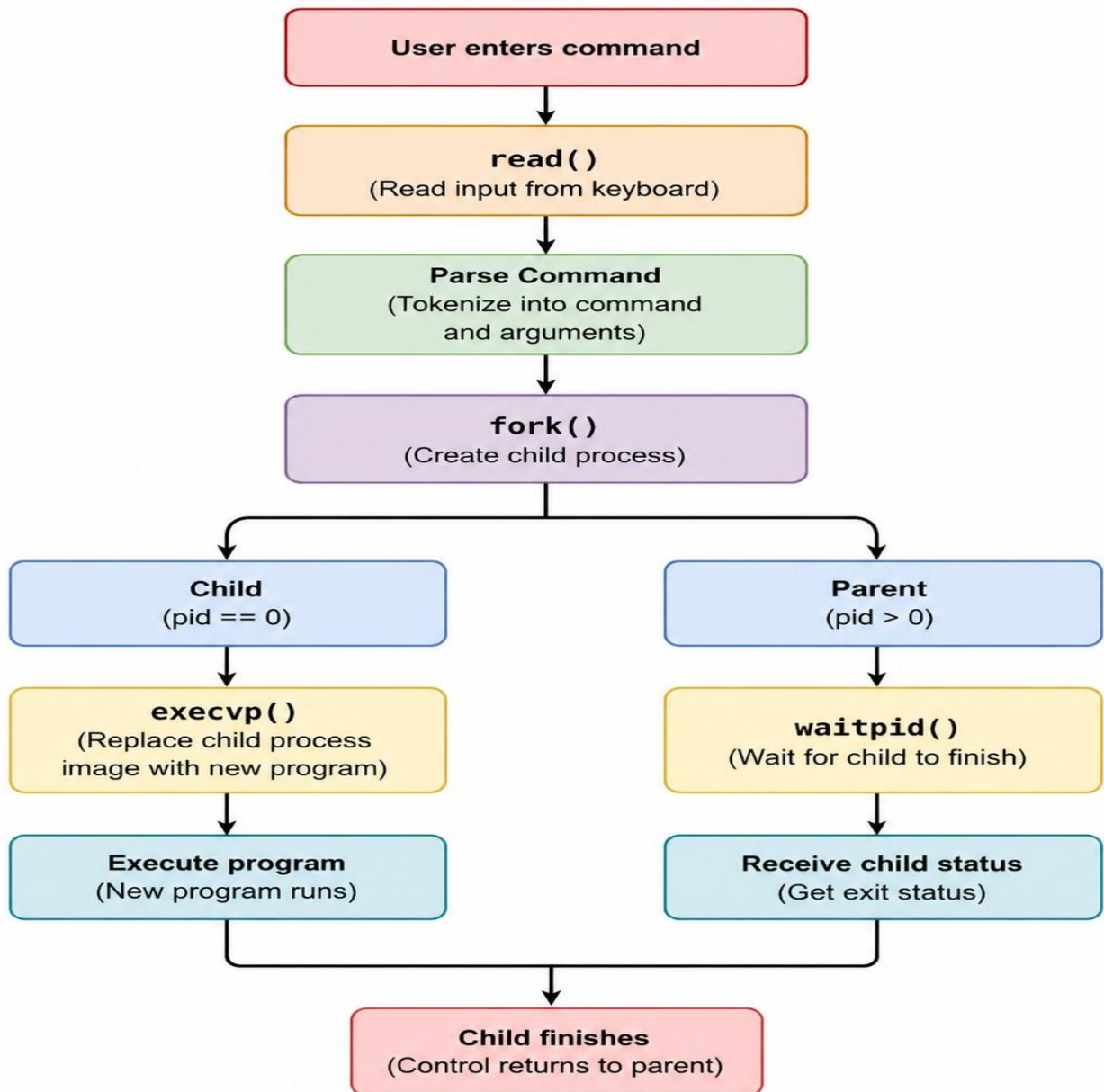
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World & Test

Child Process PID = 1821
Hello World & Test

Child process completed.
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Flow Chart:



Sample Output:

Case 1: myshell\$

ls

Parent Process PID: 1200

Created Child PID: 1201

Child Process PID: 1201

file1.txt file2.txt

Child exited with status: 0

Case 2:

myshell\$ ls @! Parent

Process PID: 82

Child Process PID: 83 Created

Child PID: 83

ls: cannot access '@!': No such file or directory

Child exited with status: 2

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <string.h>
```

```
#define MAX_INPUT 100
```

```
#define MAX_ARGS 20
```

```
int main() {
```

```
    char input[MAX_INPUT];
```

```
    while (1) {
```

```
        printf("\nmyshell$ ");
```

```
        fflush(stdout);
```

```
        if (fgets(input, sizeof(input), stdin) == NULL)
```

```
            break;
```

```
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
    continue;

if (strcmp(input, "exit") == 0)
    break;

char *args[MAX_ARGS];
int i = 0;

char *token = strtok(input, " ");

while (token != NULL && i < MAX_ARGS - 1) {
    args[i++] = token;
    token = strtok(NULL, " ");
}

args[i] = NULL;

printf("Parent Process PID: %d\n", getpid());

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    continue;
}

if (pid == 0) {
    printf("Child Process PID: %d\n", getpid());

    execvp(args[0], args);

    perror("execvp failed");
    exit(127);
}
```

```

    }
    else {
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }
}

return 0;
}

```

OUTPUT:

CASE 1:

```

chakrika@Chakrika:~/OSSP$ ./command_shell

myshell$ myshell$ ls
Parent Process PID: 1935
Created Child PID: 1938
Child Process PID: 1938
execvp failed: No such file or directory
Child exited with status: 127

```

CASE 2:

```
myshell$ myshell$ ls @l  
Parent Process PID: 1935  
Created Child PID: 1943  
Child Process PID: 1943  
execvp failed: No such file or directory  
Child exited with status: 127
```

```
myshell$ myshell$ exit  
Parent Process PID: 1935  
Created Child PID: 1944  
Child Process PID: 1944  
execvp failed: No such file or directory  
Child exited with status: 127
```